

Problem 1

In problem 3 I made a typo. It is write in my pseduocode, but I just did the cases in the proof for correctness incorrect. I should be $mid > left$ and $mid \leq left$.

Case 2: If $A[mid] > A[mid]$, the left half of the array is sorted. Specifically:

$$A[mid] < A[mid + 1] < A[mid + 2] < \dots < A[mid]$$

Since the left half is sorted, the transition point (where the maximum wraps to the minimum) cannot lie in the left half. Thus, we update the search to the right half by setting $mid \leftarrow mid + 1$. We know assume Case 2 did not happen.

Case 3: If $A[mid] \leq A[mid]$, the right half of the array is sorted. Specifically:

$$A[mid] < A[mid + 1] < A[mid + 2] < \dots < A[right] < A[mid]$$

Since the right half is sorted, the transition point must lie in the left half. Thus, we update the search to the left half by setting $right \leftarrow mid$.

Finally, if the while loop terminates without returning (i.e., when $left = right$), the range contains only one element, which must be the maximum. This is because each iteration eliminates half of the array, ensuring the maximum remains in the reduced range. When one element remains, it is guaranteed to be the maximum.

Problem 2

- (a) This problem is equivalent to the Coin Change Problem. Both problems ask for the number of ways to achieve a target sum using a given set of values, where each value has a limited number of occurrences. In the coin change problem, you are given coins of certain denominations (each coin can be used at most as many times as it appears) and you must count the number of unique collections that add up to a specified amount. Similarly, in the submultiset problem, each element in the multiset can only be used as many times as it appears, and you are tasked with counting the number of submultisets that sum to the target t . The constraints in both problems restrict the number of times an item (or coin) can be used, and the goal is to optimize the count of valid combinations that exactly match the target sum.
- (b) This problem is a reformulation of the 0–1 Knapsack Problem. In the 0–1 knapsack problem, you are given a set of items, each with a weight and a value, and the objective is to select a subset of items that maximizes the total value without exceeding a given weight capacity. In the pipe cutting problem, the pipe of length L serves as the capacity, while each available pipe length (with its associated price) corresponds to an item with a “weight” equal to its length and a “value” equal to its price. The restriction that you can sell each length only once mirrors the 0-1 constraint in the knapsack problem, where each item may be selected at most once. Thus, the problem constraints and the objective—to maximize revenue while staying within the total length—directly correspond to the weight capacity and value maximization in the 0–1 knapsack problem.

Problem 3

- (a) (i) Y 's recurrence is flawed because it fails to consider that the desired subset summing to H may be formed by selecting noncontiguous elements from S . In other words, the recurrence only "remembers" if any prefix $\{s_1, \dots, s_j\}$ has already summed to H , and if not, it simply carries forward the result from $P(i-1)$ without considering the possibility of including or excluding the current element s_i to form a new sum that might equal H . This approach overlooks the combinatorial nature of the subset-sum problem, where you must decide, for each element, whether to include it or not. Consider the set $S = \{1, 3, 2, 4\}$ where the total sum is 10, so $H = 5$. Although there exists a valid subset $\{1, 4\}$ that sums to 5, Y 's recurrence would only check contiguous prefixes and find that $P(1)$ is false (since $1 \neq 5$) and $P(2)$ is false (since $1 + 3 = 4 \neq 5$). Thus, it fails to account for the non-contiguous combination $\{1, 4\}$, leading to the incorrect conclusion that no valid partition exists.
- (ii) Look at homework 3 problem 4 part c. Let $f(i, t)$ be the recurrence from that problem where this is the value for if we can get to a sum of size t with the first i elements. The same recurrence would work here we would just use $t = \text{sum}/2$ where sum is the sum of the total array.
- (b) (i) Y 's recurrence is flawed because it assumes that the optimal subarray product in a given range can always be extended contiguously by simply multiplying an endpoint element with the best product from the subarray that excludes that endpoint. In reality, the recurrence uses $S(i+1, j)$ or $S(i, j-1)$ without guaranteeing that the subarray achieving that product actually touches the corresponding boundary ($i+1$ or $j-1$). This means that when you multiply by $A[i]$ or $A[j]$, you might be "gluing together" non-adjacent elements, producing a product that doesn't correspond to any contiguous subarray in the original array. Additionally, the special case where $S(i+1, j) \geq 1$ or $S(i, j-1) \geq 1$ forces a choice of an endpoint element too early, potentially missing a smaller product that could be formed by a proper contiguous combination.
- (ii) Define $P(i)$ to be the minimum product of any contiguous subarray ending at index i . Then the recurrence is

$$P(1) = A[1],$$

$$P(i) = \min\{A[i], A[i] \cdot P(i-1)\} \quad \text{for } i = 2, 3, \dots, n.$$

The justification is as follows: when considering a contiguous subarray ending

at $A[i]$, there are only two possibilities. Either the optimal subarray ending at i consists solely of $A[i]$ (i.e., we start fresh at index i), or it is obtained by extending the optimal contiguous subarray ending at $i - 1$ by multiplying it by $A[i]$. Since all entries are positive, the product operation is monotonic, and so if $P(i - 1) < 1$ it is beneficial to extend the subarray (as $A[i] \cdot P(i - 1) < A[i]$); conversely, if $P(i - 1) \geq 1$ then starting a new subarray at $A[i]$ is better. Finally, the overall minimum product over all contiguous subarrays is given by

$$\min_{1 \leq i \leq n} P(i).$$

This recurrence provides a valid dynamic programming solution that runs in $O(n)$ time.